

TP3: Estimation non-paramétrique

Sarah Lemler

1. Estimation d'une densité par projection

On cherche à implémenter l'estimateur $\hat{f}_D$ par projection de la densité f de nos observations $(X_1, \dots, X_n)$ sur $[a, b]$. Pour D fixé, l'estimateur par projection de la densité sur $[a, b]$ est défini par :

$$\hat{f}_D = \sum_{j=1}^D \hat{a}_j \tilde{\varphi}_j, \quad \text{avec } \hat{a}_j = \frac{1}{n} \sum_{i=1}^n \tilde{\varphi}_j(X_i)$$

où $(\tilde{\varphi}_1, \dots, \tilde{\varphi}_D)$ est une base orthonormée de $L^2([a, b])$.

1.1 Implémentation de $\hat{f}_D$ pour D fixé

1. Simuler 1000 observations selon une loi normale d'espérance 4 et de variance 25.

```
n=      #taille de l'échantillon
X<-     #simulation du modèle N(4,25)
```

2. Définir le domaine d'estimation et implémenter la vraie fonction de densité f qu'on cherchera à estimer.

```
aX=min(X)
bX=max(X)

Xplot=seq(aX,bX,0.1) # points en lesquels on calculera la fonction de densité
fvraie=             # la vraie fonction de densité
plot(Xplot,fvraie,type='l')
```

3. Implémenter les estimateurs $\hat{a}_j$ pour $j = 1, \dots, D$ et $D = 5$ dans la base trigonométrique. Il faudra effectuer une transformation pour se ramener à l'intervalle $[0, 1]$. Pour passer de l'intervalle $[0, 1]$ à un intervalle quelconque $[a, b]$, on fait la transformation suivante :

$$\tilde{\varphi}_j(x) = \frac{1}{\sqrt{b-a}} \varphi_j\left(\frac{x-a}{b-a}\right).$$

Les $(\tilde{\varphi}_j)$ définissent une base orthonormée de $L^2([a, b])$. En effet,

$$\int_a^b \tilde{\varphi}_j(u) \tilde{\varphi}_k(u) du = \int_0^1 \varphi_j(x) \varphi_k(x) dx$$

avec le changement de variable $x = \frac{u-a}{b-a}$.

```
U<-(X-aX)/(bX-aX)
XXplot<-(Xplot-aX)/(bX-aX)

# base trigonométrique
a1f<-
```

```
a2f<-sqrt(2)*mean(cos(2*pi*U))/sqrt(bX-aX)
a3f<-
a4f<-
a5f<-
```

4. Donner l'estimateur par projection de la densité pour $D = 3$ et $D = 5$.

```
estf<- #D=3
```

```
estfbis<- #D=5
```

5. Coder la base d'histogramme pour $D = 10$.

```
# Base d'histogrammes D=10
D=
hataf<-rep(0,D)
for (j in 1:10){
  T=((U>=(j-1)/D)&(U<j/D))
  hataf[j]<-sqrt(D/(bX-aX))*mean(T)
}
```

6. Donner l'estimateur par histogramme de la densité f .

```
fhisto<-rep(0,length(Xplot))
for (j in 1:D){
  TT<-((Xplot>=aX+(j-1)*(bX-aX)/D)&(Xplot<aX+j*(bX-aX)/D))
  fhisto<-fhisto+hataf[j]*TT
}
```

7. Tracer la vraie densité et afficher sur le même graphe **estf**, **estfbis** et **fhisto**.

```
plot(Xplot,fvraie,type="l",las=1,col='red')
lines(Xplot,...,col="blue")
...
legend("topright",c("Vraie","Trigo D=3","Trigo D=5","HistoD=10"),
      col=c("red","blue","green","cyan"),lty=1,bty='n')
```

1.2 Sélection de modèle

1. Simuler un échantillon de taille 1000 de loi $\mathcal{N}(4, 25)$.

```
n= #taille de l'échantillon
X<- #simulation du modèle N(4,25)

aX=
bX=
Xplot=
fvraie=

U<-
XXplot<-
```

2. Implémenter les estimateurs $\hat{a}_j$ dans la base trigonométrique dans le cas général ainsi que le contraste utilisé pour faire de la sélection de modèle :

$$\hat{D} = \arg \min_D \left\{ -\|\hat{f}_D\|^2 + K\Phi_0^2 \frac{D}{n} \right\} = \arg \min_D \left\{ -\sum_{j=1}^D \hat{a}_j^2 + K\Phi_0^2 \frac{D}{n} \right\}$$

```

d<-floor(sqrt(n)) # dimension D=2d+1<= 2*[sqrt(n)]+1 arbitraire
af<-rep(0,2*d+1)
gamma<-rep(0,d+1)
pen<-rep(0,d+1)
estf<-matrix(0,length(Xplot),d+1)
kappa=0.1

# Calcul des contrastes gamma=-sum(hata_j^2)_j=1^D et des pénalités Kappa*D/n
af[1]<-
estf[,1]<-
gamma[1]=
pen[1]<-

for (j in 1:d){
  af[2*j]<-
  af[2*j+1]<-
  estf[,j+1]<-
  gamma[j+1]<-
  pen[j+1]<-
}

```

3. Sélection de modèle.

```

# sélection du modèle
hatd<-which(...)
estimfinal<-

```

4. Représenter l'estimateur adaptatif sur un graphe et l'ensemble des estimateurs candidats sur une autre graphe.

```

par(mfrow=c(1,2))
plot(Xplot,fvraie,type="l",col="red",las=1)
lines() # estimateur final

plot(Xplot,fvraie,type="l",col="red",las=1,ylim=c(0,max(estf)))
for (j in 1:d){
  lines() # l'ensemble des estimateurs candidats
}

```

2. Estimateur à noyau d'une densité

On cherche à implémenter l'estimateur à noyau $\hat{f}_h$ de la densité f . Pour une fenêtre h fixée, l'estimateur à noyau de f est défini par

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - X_i}{h}\right)$$

2.1 Estimateur à fenêtre fixée

1. Simuler un échantillon de 1000 lois normales centrées réduites et implémenter la densité f de la loi normale centrée réduite (qui correspond à la vraie densité).

```

n=1000
X=rnorm(n,0,1)

a=min(X)
b=max(X)
Xplot=seq(a,b,0.1)
fvraie=

```

2. Calculer un estimateur à noyau avec un noyau gaussien et d'Epanechnikov et une fenêtre fixée ($h = 1$).

Noyau gaussien : $K(x) = \frac{1}{\sqrt{2\pi}} \exp(-x^2/2)$

Noyau d'Epanechnikov : $K(x) = \frac{3}{4}(1-x^2)\mathbf{1}_{|x|\leq 1}$

```

h=      #fenêtre fixée
fest<-rep(0,length(Xplot))
fest2<-rep(0,length(Xplot))
Gaus<-matrix(0,length(Xplot),n)
EPA<-matrix(0,length(Xplot),n)

for (i in 1:length(Xplot)){
  for (j in 1:n){
    U=(Xplot[i]-X[j])/h
    Gaus[i,j]<- # noyau gaussien
    T<-(abs((Xplot[i]-X[j])/h)<=1)
    EPA[i,j]<-  # noyau d'Epanechnikov
  }
}

fest<- #estimateur avec noyau gaussien
fest2<- #estimateur avec noyau d'Epanechnikov

```

3. Représenter la vraie fonction et les deux estimateurs **fest** et **fest2**

```

plot(Xplot,fvraie,type='l',las=1,col='red')
lines(Xplot,...,col="blue",lty=2)
lines(Xplot,...,col="green",lty=4)
legend(...)

```

2.1 Sélectionner la fenêtre par validation croisée.

$$\hat{J}(h) = \int \hat{f}_h^2(x) dx - \frac{2}{n} \sum_{i=1}^n \hat{f}_{h,-i}(X_i) \quad \text{avec} \quad \hat{f}_{h,-i}(X_i) = \frac{1}{(n-1)h} \sum_{\substack{j=1 \\ j \neq i}}^n K\left(\frac{X_i - X_j}{h}\right)$$

```

K=50
ab<-seq(a,b,length.out=K)
fvraie=exp(-ab^2/2)/sqrt(2*pi)

M=40 # nombre de fenêtre qu'on va considérer : h=k/M avec k=1,...,M
estimf<-matrix(0,M,K) # on va stocker dans une matrice les valeurs des
#estimateurs pour chacun des points ab en colonne et chacune des fenêtres en ligne
h<-rep(0,M)

# 1ère partie du critère : hatf^2

```

```

for (k in 1:M){
  h[k]<-k/M
  #noyau d'Epanechnikov
  Tt<-(abs(((matrix(X,n,K,byrow=F)-t(matrix(ab,K,n,byrow=F)))/h[k]))<=1)
  #remplissage de la matrice par colonne
  estimf[k,]<-apply((3/4)*(1-((matrix(X,n,K,byrow=F)-
                                t(matrix(ab,K,n,byrow=F)))/h[k])^2)*Tt,2,mean)/h[k]
}

# 2ème partie du critère : sum_i != j
Mat<-array(0, dim=c(n,n,K))
R<-diag(1,n,n)
h<-rep(0,M)
for (k in 1:M){
  h[k]<-k/M
  for (i in 1:n){
    for (j in 1:n){
      aa<-(abs((X[i]-X[j])/h[k]))<=1)
      Mat[i,j,k]<-(3/4)*(1-((X[i]-X[j])/h[k])^2)*aa/h[k]-(3/4)*R[i,j]/h[k]
    }
  }
}

# Critère complet
Crit<-rep(0,M)
for (k in 1:M){
  Crit[k]<-sum(estimf[k,]^2)*(b-a)/K-2*sum(apply(Mat[, ,k],1,sum))/(n*(n-1))
}

# Sélection de la fenêtre
estimfinal<-rep(0,K)
hath<-which(...)
estimfinal<-

par(mfrow=c(1,2))
plot(ab,fvraie,type="l",las=1,col="red")
lines(ab,...)

plot(ab,fvraie,type="l",las=1,col="red",ylim=c(0,max(estimf)))
for (k in 1:M){
  lines(ab,...)
}

```

3. Estimateur d'une fonction de régression par moindres carrés

3.1 Estimation sans sélection de modèle

```

rm(list=objects()); graphics.off()
n<-1000

```

On commence par simuler le modèle : $Y_i = m(X_i) + \varepsilon_i$ avec $\varepsilon_i \sim \mathcal{N}(0, 0.5)$, $X_i \sim \mathcal{N}(0, 1)$ pour $i = 1, \dots, n$ et

$$m(x) = x^4.$$

```
#simulation du modèle
eps<- # epsilon~N(0,0.5)
X<- # X~N(0,1) # observations
mX<- # la fonction non linéaire à estimer
# Autres fonctions que l'on pourra tester
# mX=aX+b
# mX=exp(X)
# mX=X^2*exp(-3*abs(X))
Y<-
```

On définit ensuite la vraie fonction à estimer en des points choisis.

```
# les vraies fonctions
aX<-min(X)
bX<-max(X)
Xplot<-seq(aX,bX,length.out=100)

mVraie<-
# mVraie=aXplot+b
# mVraie=exp(Xplot)
# mVraie=Xplot^2*exp(-3*abs(Xplot))
```

Estimer la fonction de régression sur l'intervalle $[a, b]$ à l'aide des moindres carrés en considérant une base trigonométrique de taille $D = 2 * d + 1$ et $d = 4$.

```
# estimation en polynômes trigonométriques
U<-(X-aX)/(bX-aX) #pour se ramener à [0,1] (observations)
XXplot<-(Xplot-aX)/(bX-aX) #(points en lesquels on va calculer l'estimateur)
d<-4

phi<-matrix(0,length(U),2*d+1) # de taille n*D
phi[,1]<-
phiplot<-matrix(0,length(XXplot),2*d+1) # de taille length(XXplot)*D
phiplot[,1]<-

for (j in 1:d){
  phi[,2*j]<-
  phi[,2*j+1]<-
  phiplot[,2*j]<-
  phiplot[,2*j+1]<-
}

library(MASS) #pour la fonction ginv (solve ne marche pas ici)
# hat_a_D<-G_D^-1*Phi^TY/n=(1/n^2)*Phi(Phi)^T Phi Y
hataf<-
estmX<-
```

Représenter graphiquement la vraie fonction et la fonction estimée.

3.2 Moindres carrés adaptatifs (sélection du modèle)

On commence par simuler le modèle : $Y_i = m(X_i) + \varepsilon_i$ avec $\varepsilon_i \sim \mathcal{N}(0, 0.5)$, $X_i \sim \mathcal{N}(0, 1)$ pour $i = 1, \dots, n$ et $m(x) = x^2$.

```
rm(list=objects()); graphics.off()
# Estimation sans sélection de modèle
n<-1000

#simulation du modèle
eps<-0.5*rnorm(n) # epsilon~N(0,sqrt(0.5))
X<-rnorm(n) # X~N(0,1) # observations
mX<- X^2 # la fonction non linéaire à estimer
# mX=aX+b
#mX=exp(X)
# mX=X^2*exp(-3*abs(X))
```

```
Y<-mX+eps
```

```
# les vraies fonctions
aX<-min(X)
bX<-max(X)
Xplot<-seq(aX,bX,length.out=100)

mVraie<-Xplot^2
# mVraie=aXplot+b
# mVraie=exp(Xplot)
# mVraie=Xplot^2*exp(-3*abs(Xplot))
```

Pour sélectionner le meilleur modèle, il va falloir calculer

$$\hat{D} = \arg \min_{D \in \{1, \dots, \lfloor \sqrt{n} \rfloor\}} \{-\|\hat{f}_D\|_n^2 + \text{pen}(D)\}$$

avec $\|t\|_n^2 = \frac{1}{n} \sum_{i=1}^n t(X_i)$ et $\text{pen}(D) = K \frac{\sigma_\varepsilon^2 D}{n}$ avec $K = 5/2$. Compléter le code suivant.

```
# estimation en polynômes trigonométriques
U<-(X-aX)/(bX-aX) #pour se ramener à [0,1] (observations)
XXplot<-(Xplot-aX)/(bX-aX) #points en lesquels on va calculer l'estimateur

dmax<-

phi<-matrix(0,length(U),2*dmax+1) # de taille n*D
phi[,1]<-
phiplot<-matrix(0,length(XXplot),2*dmax+1) # de taille length(XXplot)*D
phiplot[,1]<-

for (j in 1:dmax){
  phi[,2*j]<-
  phi[,2*j+1]<-
  phiplot[,2*j]<-
  phiplot[,2*j+1]<-
}

gammapen<-rep(0,dmax) # critère à minimiser
for (k in 1:dmax){
  hataf<-ginv(t(phi[,1:(2*k+1)]))%*%phi[,1:(2*k+1)]/n)%*%t(phi[,1:(2*k+1)])%*%Y/n
  ## hat_a_D<-G_D^{-1}Phi_D^TY/n=(1/n^2)Phi_DPhi_D^TPhiDY
  estmX<-phi[,1:(2*k+1)]%*%hataf
```

```

# hat_f<-t(Phi_D)%*%hat_a_D calculé en les observations
  gammapen[k]<-
}

dopt<-
hatafD<-
estmX<-

```

Représenter la vraie fonction et l'estimateur final obtenu sur un même graphique.